Harini Buddaluru

Professor Wallisch

Fundamentals of ML

14 May 2025

This project tackled the task of predicting music genre from Spotify's audio features using classification models. Since genre classification was inherently a multi-class problem (with 10 different classes), I tried a variety of modeling approaches and preprocessing strategies to improve AUROC performance. In particular, I focused on data preparation, dimensionality reduction, feature selection, clustering, and choosing the right model architecture and hyperparameters. The final AUC was 0.9318 using HistGradientBoostingClassifier, and I walked through how I got there. I decided also not to use neural nets because they worked best with unstructured data, and this was neat tabular data for which it may have worked well but was not necessary. Better suited methods that were more interpretable and faster to train existed.

I began with exploring the data, identifying five completely null rows which were dropped. Tempo was an object and was converted to be numerical. Then I checked for invalid/missing values, finding ~15k for instrumentalness and ~5k for tempo and duration. So I also inspected to see if these were commonly occurring together or if there were any patterns there. Fortunately, I found no real pattern happening with relation to this missingness. It appeared random, so I had no cause for concern thus far for moving forward with imputation.

Then I did some general exploration to understand the data and found the classes were extremely balanced, which was good for the classification model to learn all genres equally well. The distributions were a little more interesting. Most of the numerical features were not normally distributed, many with extreme skews. Beyond scaling, I wanted to use tree-based models because I had mixed feature types. Tree-based models natively handle categorical features well without needing any linear assumptions. Nonlinear feature interactions between these features and genre also likely exist, just based on my rudimentary knowledge of music, and again this is natively supported in tree-based models. They are also more robust to outliers and skew which are present in this dataset.

There were also mostly weak correlations across the board, with some obvious exceptions such as energy and acousticness or loudness and acousticness. I was not concerned about collinearity here. Anyway, PCA removes redundancy when it creates orthogonal components, and also tree-based models select splits based on information gain, so they are already going to ignore redundant features that do not add predictive power.

The missing tempo was pretty evenly split across genre and the key features for songs with and without it were also very similar. Essentially, there were no concerning patterns and the missingness appeared random, so I could safely impute these values without introducing bias

into the dataset. The same was true for duration. I found more concerning issues with instrumentalness. Many more values were missing for hip-hop than for classical, to start. Missingness wasn't random and was probably reflecting some structural or cultural patterns in the data. For example, hip-hop usually had vocals, so low instrumentalness would make sense and maybe got recorded as 0 or left blank entirely. The songs missing instrumentalness actually had higher popularity on average as well, which felt important and suggested there might have been a real signal tied to those missing values. That was why instead of doing global imputation I opted for imputing within each genre where the context stayed consistent, and I did this for all three columns.

I first tried to impute all three columns with regression. To this extent, I imputed each value with all the rows of that dataset from that same genre, to preserve the context I discussed earlier and hopefully get more reasonable imputations. This was especially tricky for values like instrumentalness, which had far more missing values in some genres than others. The distributions for instrumentalness before and after imputation were very different. Tempo was also quite different, whereas instrumentalness didn't see a huge change. A bunch of imputed instrumental values also ended up being below 0. As for the correlations I saw no concerning differences although correlations for duration and other columns generally went up a tad. I made sure to impute all the columns at once so the imputations were not building off each other, and I saw no concerning correlations between imputed columns, so that method was successful.

Because of that issue I decided to try imputing only duration with the Linear Regression and using genre-based random sampling to impute tempo and instrumentalness. Tempo has a good range of all possible values to sample from and instrumentalness is on a scale from [0,1] all of which are present in the dataset. I tried to impute by global median and genre based medians in previous experiments and found those methods weren't great for the distributions either. In this case, also since I am never using genre to predict or impute any of these values, I am making sure to prevent data leakage. Splitting the data by genre before imputation ensures the imputation is done with genre specific information but without leakage.

Instrumentalness still looked quite different after imputation but this is to an extent expected since there's 15k missing values from a dataset of only 50k (very large chunk). I originally considered dropping instrumentalness altogether but decided against it in the end since the dataset is so high dimensional I figured a tree-based model would not give it much importance anyway in cases where it harmed the classification prediction. I saw a significant improvement in the distributions for tempo, and there were no invalid values with this method. This imputation process preserved the original structure and variability of the data, rather than artificially smoothing it out or introducing bias. That's especially important here since we're using the imputed features (like tempo and instrumentalness) as inputs in classification and clustering models. If the distributions had shifted significantly, the models might have learned patterns that

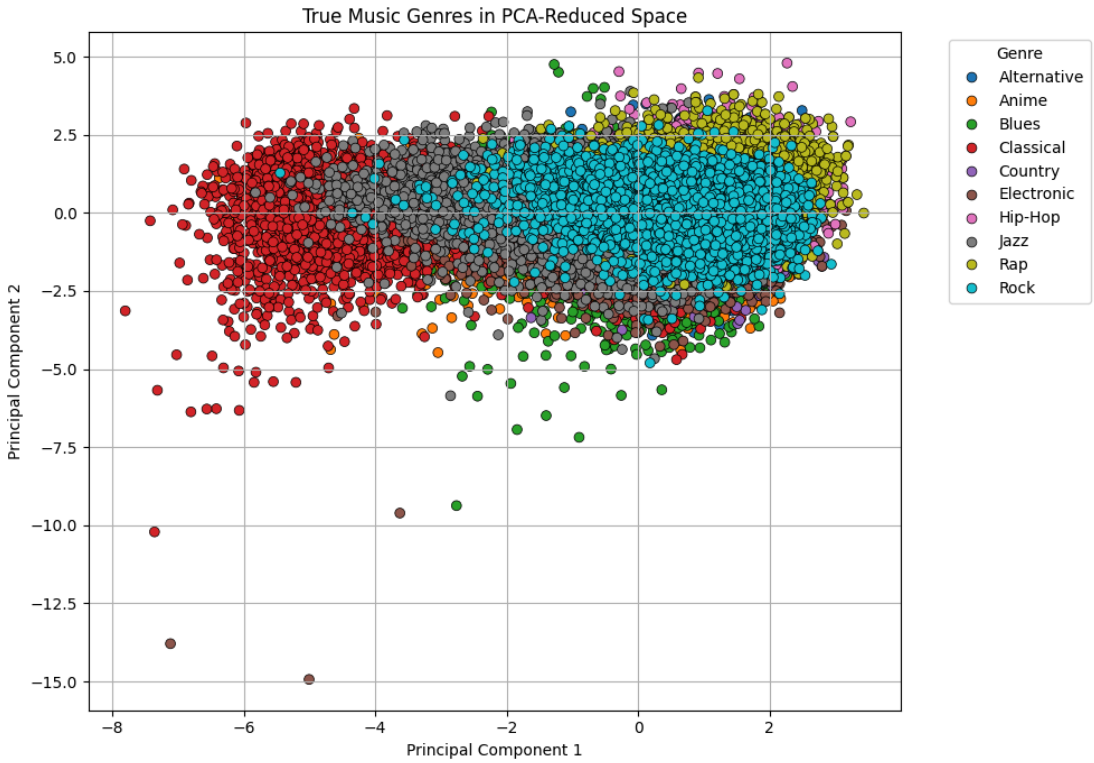
don't actually exist in the original data. I also saw no large or concerning changes of any sort in the correlation map before and after imputation, which was again good.

Then I did all the formatting to get the train and test splits as specified in the spec sheet, including standard scaling and encoding labels for categorical columns. For dimensionality reduction, I chose PCA because it keeps the global structure of the data and gives you actual principal components that are linear combinations of the original features that can be traced back so I can understand and interpret, unlike t-SNE or other methods. PCA also works well when features are on different scales after standardization, so I felt based on what I learned about my data it would be a good fit. n_components was 0.95 so I could keep enough information to still train accurate models without losing too much variation. After PCA I had 9 dimensions. The first two PCs explained very little of the variation though, 0.33 and then 0.12 respectively so combined it was still less than half. This made it not great for visualizing the clusters and such.

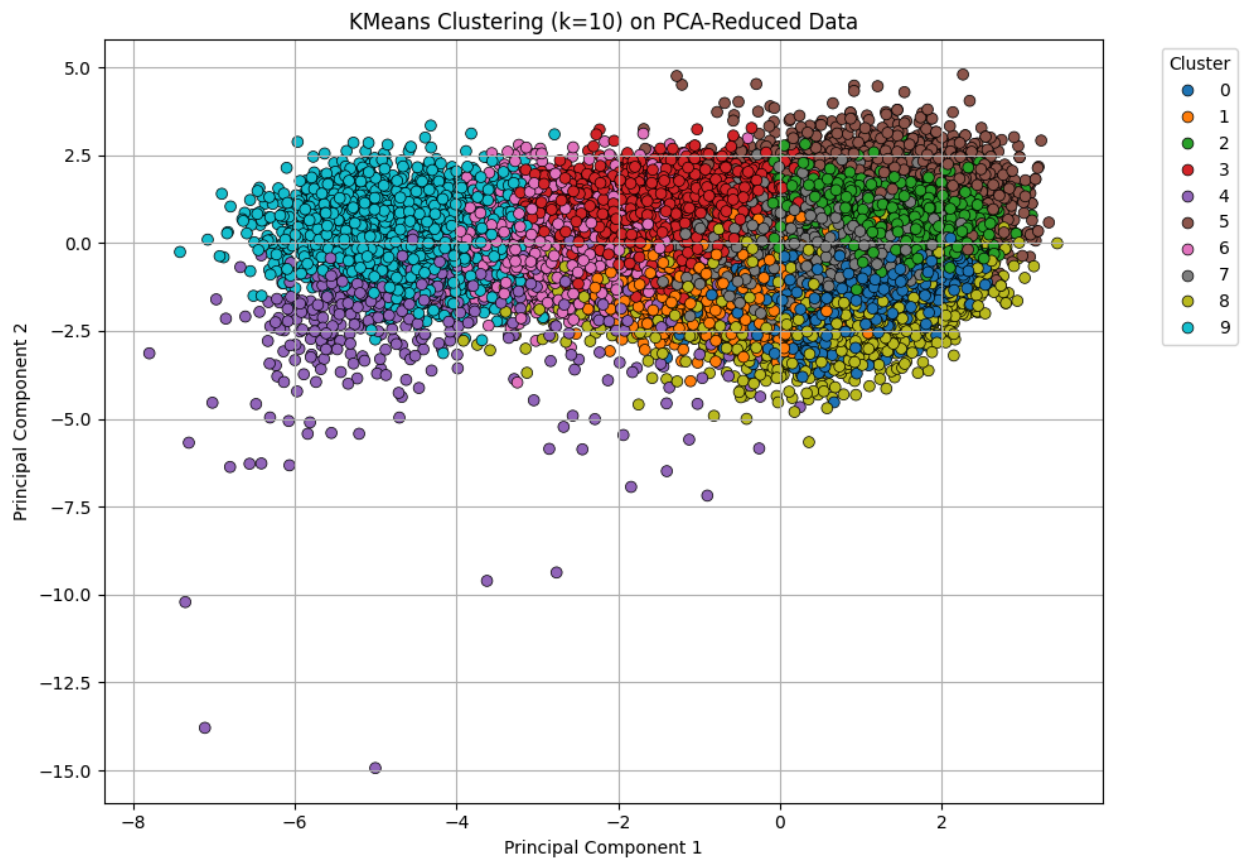
PC1, which explained the most variance, was dominated by loudness, energy, and acousticness. These all intuitively go together and probably capture a kind of "intensity" or production quality axis across songs. PC2 had more rhythmic and structural elements like danceability, tempo, and duration, while also pulling in popularity. PC3 surprised me a bit since it’s heavily influenced by speechiness and liveness, which makes me think it might be picking up on performance or vocal presence, maybe distinguishing spoken-word or live tracks. Across PC4 and PC5, duration keeps showing up, often alongside valence and tempo, which tells me those features contribute meaningful variation across genres.

I ran silhouette scores and plotted the curve to see if the elbow was at k=10. If it was, that would suggest that PCA and clustering is picking up on some patterns genuinely present in the data (as we have 10 genres). Unfortunately, the curve was very wonky, increasing and decreasing with no clear elbow at k=10, indicating that the PCA=reduced space may not separate the genres in a way that's meaningful for unsupervised clustering.

Here I visualize the genres in 2D using the first two PCs. Clearly, we need at least a third dimension to have more interpretable visual separation as we can see how the genres are overlaid on each other in this 2D space.



And as suspected earlier, the KMeans clustering is not picking up on meaningful structures in the data to cluster the genres in this 2D space as we compare the real clusters above with what KMeans finds and see that it has identified nothing meaningful. KMeans clustering with PCA in 2D is not useful.



Then I run some logistic regression with a couple different regularization values and on the raw unscaled data, raw scaled data, and PCA data to see which performs best. This serves as the baseline to compare our tree-based models too since Logistic Regression is pretty simplistic in its linear assumptions. I found that scaling and dimensionality reduction significantly improved performance for logistic regression, which makes sense since it's sensitive to scale and doesn't work well with nonlinear relationships. Scaling also smoothens out the optimization landscape and makes it so the LR can't handle as much regularization. This is why there's so little regularization in scaled and PCA than unscaled (10 vs. 0.01). AUC of 0.8837 is also quite strong, so it indicates the preprocessing was good.

```
Raw (Unscaled) → AUROC: 0.7755, Best C: 0.01
Raw (Scaled) → AUROC: 0.8872, Best C: 10
PCA-Reduced → AUROC: 0.8837, Best C: 10
```

Next I try a default random forest classifier and already find AUC improves to 0.917. For the grid search I picked hyperparameters that aimed to balance model complexity, performance, and generalizability. I found the best performance came from n_estimators of 200, max_depth of 20, min_samples_split of 5, and max_features as 'sqrt'. This yielded an AUC of 0.9238.

This makes sense since 200 trees would give the model enough diversity to reduce variance without becoming too slow or redundant, capping depth at 20 helps avoid overfitting but you can still capture meaningful structure in the data, needing 5 samples to split regularizes the model a bit by preventing it from relying on noise or overly specific patterns, and 'sqrt' is the standard method of classification that probably helped here as it introduced variation between trees, boosting ensemble strength.

I also attempted to fit this same tree on the raw scaled data and the PCA-reduced data and found slightly lower performance at AUCs of 0.9237 and 0.9091 respectively. Well, scaled and unscaled essentially had identical performance but with PCA-reduced it was considerably lower.

I suspect this is because Random Forests don't benefit much from PCA since they already handle high-dimensional, non-linear data quite well without needing orthogonal transformations. PCA reduces dimensionality by combining features based on variance, but that can actually remove useful signals for tree-based models. RFs are also are scale-invariant.

Next I tried GBDT, in this case HistGradientBoostingClassifier which is a faster version than regular GBDT since it bins continuous features into discrete histograms, reducing the number of candidate splits it has to evaluate. That means it pairs well with this structured classification problem.

To this extent I tried fitting a default HistGBDT on raw unscaled data and got an AUC of 0.9318, higher than our best RF and the best performance overall.

Afterwards I also defined a hyperparameter space to search that again, aimed to find a model that would balance model complexity, performance, and generalizability. The best combination I found was learning_rate of 0.05, max_iter of 100, max_depth of None and l2_regularization of 1.0.

These were likely the best performers because learning rate of 0.05 lets the model learn gradually, preventing overfitting while still capturing meaningful patterns and also smooths out the influence of individual trees. max_iter of 100 lets the model have enough boosting rounds to converge without overfitting too much, while max_depth of None lets each tree fit complex interactions if it so desires. This is good since we have a lot of features with different types of relationships (linear or nonlinear). The l2_regularization of 1.0 gives some constraint on the leaves so we don't get predictions that are too sharp so this also helps with generalization (keeps us from relying too much on noise).

However, what's really interesting is that despite all the values in this grid search being present for it to recreate the default tree with 0.9318 AUC, it ended up choosing a combination with a microscopically lower AUC of 0.9315. I think this difference is due to the introduction of 3-fold CV when doing the grid search so there is some stochasticity which led to this. Basically, I think it is safe to say both have essentially the same, strong performance.

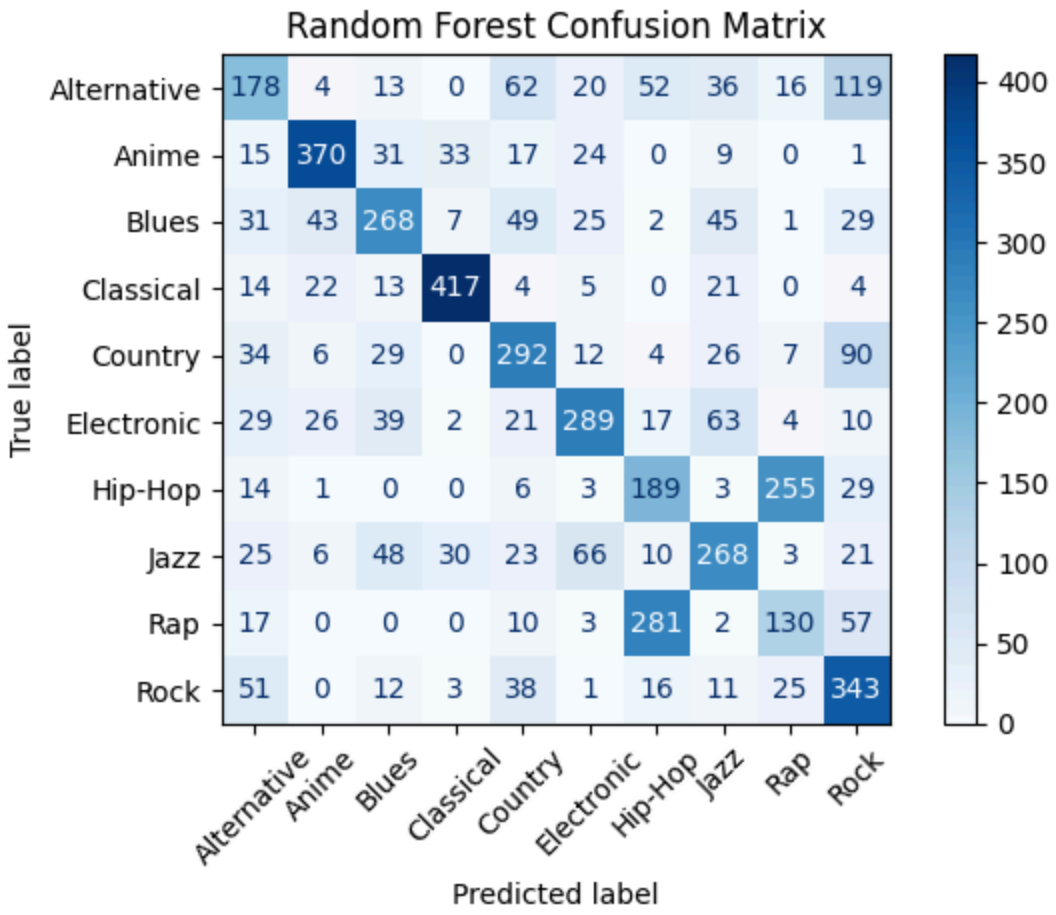
An AUC of 0.93 indicates that the model is able to predict genre really well based on the raw, unscaled features. With scaling it's 0.9311 (essentially identical), which makes sense as HistGBDT is mostly scale-invariant. The PCA-reduced had a lower AUC of 0.9137 for the same reason RF has lower performance with PCA-reduced. Essentially, dimensionality reduction is not helpful with tree based ensemble methods for this use case.

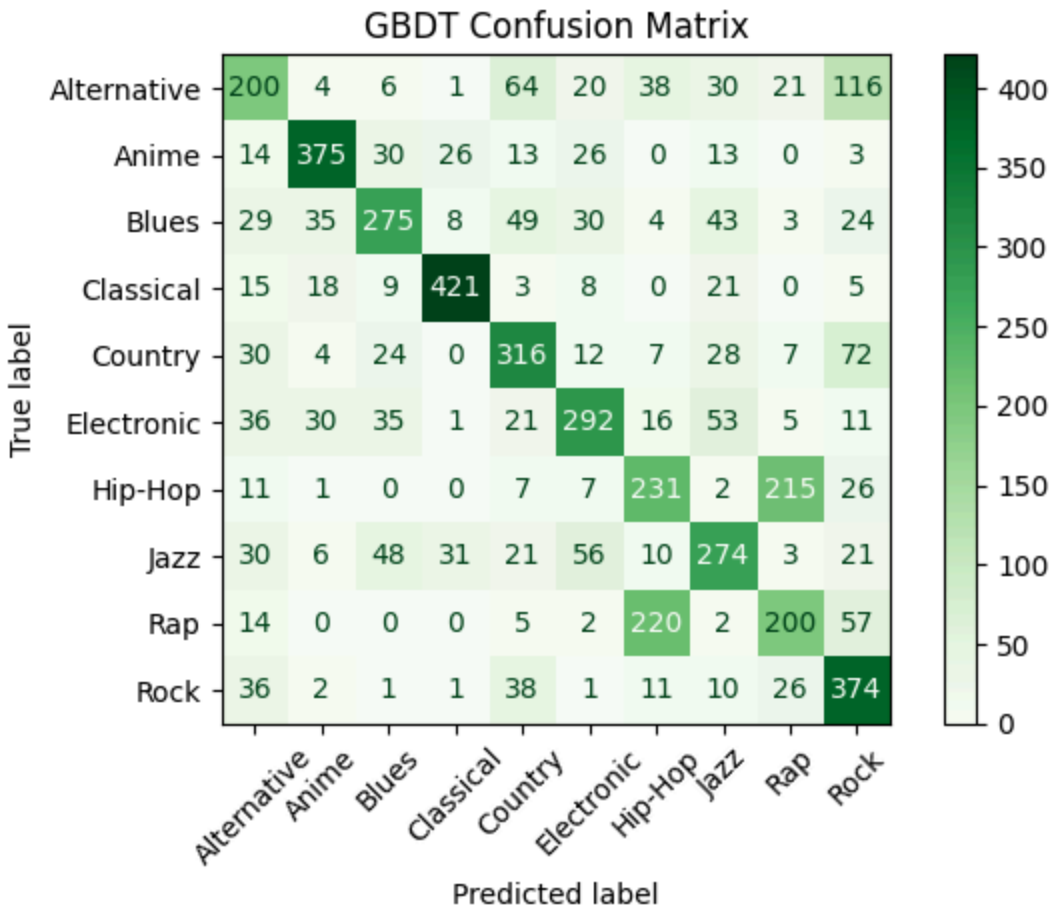
RF is a bagging method where many trees are trained independently on random subsets of the data before averaging their outputs. HistGBDT is a boosting method where trees are built sequentially and each new tree is learning and correcting the errors of its predecessor. In this case I think HistGBDT did better because likely this data has subtle patterns and some genres are definitely harder to classify than others.

There's surely quite a bit of variation in the dataset in general (we saw in the earlier PCA some genres such as classical are very spread across these features). Boosting is better at capturing these types of things and the binning probably helped to smooth out noisy features too. So, I think it was able to find a slightly better point in the bias vs. variance tradeoff than RF. That being said, let's look below at how each performed on different genres.

```
AUROC by genre:
Alternative  | RF AUC: 0.8549 | GBDT AUC: 0.8732
Anime        | RF AUC: 0.9673 | GBDT AUC: 0.9712
Blues        | RF AUC: 0.9053 | GBDT AUC: 0.9124
Classical    | RF AUC: 0.9766 | GBDT AUC: 0.9798
Country      | RF AUC: 0.9295 | GBDT AUC: 0.9337
Electronic   | RF AUC: 0.9274 | GBDT AUC: 0.9282
Hip-Hop      | RF AUC: 0.9092 | GBDT AUC: 0.9277
Jazz         | RF AUC: 0.9205 | GBDT AUC: 0.9254
Rap          | RF AUC: 0.9011 | GBDT AUC: 0.9211
Rock         | RF AUC: 0.9365 | GBDT AUC: 0.9467
```

HistGBDT has higher performance across the board than RF. Also the genres with the worst performance are alternative and blues. Alternative definitely makes sense since that's a very varied genre, and Anime and Classical being the easiest to classify also makes sense since they have very distinct styles. There are no concerning issues here. Let’s also look below at what types of misclassification these models are making.


Random Forest Confusion Matrix
True label
Predicted label
Alternative
Anime
Blues
Classical
Country
Electronic
Hip-Hop
Jazz
Rap
Rock
178 4 13 0 62 20 52 36 16 119
15 370 31 33 17 24 0 9 0 1
31 43 268 7 49 25 2 45 1 29
14 22 13 417 4 5 0 21 0 4
34 6 29 0 292 12 4 26 7 90
29 26 39 2 21 289 17 63 4 10
14 1 0 0 6 3 189 3 255 29
25 6 48 30 23 66 10 268 3 21
17 0 0 0 10 3 281 2 130 57
51 0 12 3 38 1 16 11 25 343
400
350
300
250
200
150
100
50
0



We see rock and alternative get confused a lot, and so do hip-hop and rap, in both models. I think these are pretty reasonable errors. Genre classifications are blurry even to me. Sometimes it really is more of a spectrum. Concerning errors such as classical and hip-hop don't really exist, so that's a good sign. Just to make it simpler, there's another method of identifying these below.

```
Random Forest Misclassifications:
True         Predicted_RF
Rap          Hip-Hop         281
Hip-Hop      Rap             255
Alternative  Rock            119
Country      Rock             90
Jazz         Electronic       66
Electronic   Jazz             63
Alternative  Country          62
Rap          Rock             57
Alternative  Hip-Hop          52
Rock         Alternative      51
Name: count, dtype: int64

GBDT Misclassifications:
True         Predicted_GBDT
Rap          Hip-Hop           220
Hip-Hop      Rap               215
Alternative  Rock              116
Country      Rock               72
Alternative  Country            64
Rap          Rock               57
Jazz         Electronic         56
Electronic   Jazz               53
Blues        Country            49
Jazz         Blues              48
Name: count, dtype: int64
```

We can see across the board that HistGBDT is making fewer misclassifications of all these types than the Random Forest, again proving itself to be the best performer.